

## Chapter Overview

Chapter 04 covers JPA/Hibernate as used in ShopVerse: entity mapping, relationship types, cascade/fetch strategies, JPQL vs native SQL, N+1 problem and fixes, Hibernate caching layers, optimistic vs pessimistic locking, schema evolution with Flyway, and performance tuning.

| Sub-Chapter | Topic                                     | Key Class/Annotation                                         |
|-------------|-------------------------------------------|--------------------------------------------------------------|
| 04-01       | Entity Mapping & Column Types             | @Entity, @Column, @Embeddable, @MappedSuperclass, BaseEntity |
| 04-02       | Relationships – One-to-Many / Many-to-One | @OneToMany, @ManyToOne, @JoinColumn, mappedBy                |
| 04-03       | Relationships – Many-to-Many & One-to-One | @ManyToMany, @OneToOne, @JoinTable                           |
| 04-04       | Fetch Strategies & N+1 Problem            | LAZY vs EAGER, JOIN FETCH, @EntityGraph, @BatchSize          |
| 04-05       | JPQL, Criteria API & Native SQL           | @Query, Specification, CriteriaBuilder, nativeQuery          |
| 04-06       | Transactions & Locking                    | @Version (optimistic), PESSIMISTIC_WRITE, Isolation levels   |
| 04-07       | Hibernate Caching                         | L1 Cache (Session), L2 Cache (Ehcache/Redis), Query Cache    |
| 04-08       | Schema Migration – Flyway                 | V__*.sql scripts, repair, baseline, validate                 |
| 04-09       | Performance Tuning & Batch Operations     | Batch inserts, projections, statistics, slow-query log       |
| 04-10       | Hibernate Envers – Auditing               | @Audited, AuditReader, revision history tables               |
| 04-11       | Spring Data Specifications & Querydsl     | Complex dynamic queries, type-safe predicates                |
| 04-12       | Inheritance Mapping Strategies            | SINGLE_TABLE, JOINED, TABLE_PER_CLASS                        |

## Base Entity – Shared Auditing Fields

```
@MappedSuperclass @EntityListeners(AuditingEntityListener.class)
public abstract class BaseEntity {
    @Id @GeneratedValue(strategy=SEQUENCE, generator="shopverse_seq")
    @SequenceGenerator(name="shopverse_seq", sequenceName="shopverse_id_seq",
        allocationSize=50) // fetch 50 IDs at once → reduces DB round-trips
    private Long id;
    @CreatedDate @Column(updatable=false) private Instant createdAt;
    @LastModifiedDate private Instant updatedAt;
    @CreatedBy @Column(updatable=false) private String createdBy;
    @LastModifiedBy private String updatedBy;
    @Version private Integer version; // optimistic locking version counter
}
```

## @Column Mapping Rules

| Java Type           | Annotation                          | DB Column Type         | Notes                                       |
|---------------------|-------------------------------------|------------------------|---------------------------------------------|
| String              | @Column(length=255)                 | VARCHAR(255)           | Default length; override for large text     |
| String (long)       | @Column(columnDefinition="TEXT")    | TEXT                   | Product description, review body            |
| BigDecimal          | @Column(precision=19,scale=4)       | NUMERIC(19,4)          | All monetary amounts in ShopVerse           |
| Instant             | @Column(name="created_at")          | TIMESTAMPTZ (Postgres) | UTC timestamps with tz offset               |
| Enum                | @Enumerated(STRING)                 | VARCHAR                | Never ORDINAL – breaks on enum reorder      |
| boolean             | @Column(name="is_active")           | BOOLEAN                | Prefix with is_; default via @ColumnDefault |
| long[] / List<Long> | @ElementCollection @CollectionTable | Separate child table   | Tag IDs, phone numbers                      |
| JSON/Map            | @JdbcTypeCode(SqlTypes.JSON)        | jsonb (Postgres)       | Hibernate 6 native JSON mapping             |

## @Embeddable Value Objects

```
@Embeddable public class Address {
    @Column(name="street") private String street;
    @Column(name="city") private String city;
    @Column(name="state") private String state;
    @Column(name="postal") private String postalCode;
    @Column(name="country") private String country;
}

@Entity @Table(name="customers") public class Customer extends BaseEntity {
    @Embedded
    @AttributeOverrides({
        @AttributeOverride(name="street", column=@Column(name="billing_street")),
        @AttributeOverride(name="city", column=@Column(name="billing_city")),
        @AttributeOverride(name="postalCode", column=@Column(name="billing_postal"))
    })
    private Address billingAddress;
}
```

## ID Generation Strategies Comparison

| Strategy | Annotation                         | Mechanism                          | Throughput | ShopVerse                     |
|----------|------------------------------------|------------------------------------|------------|-------------------------------|
| AUTO     | @GeneratedValue                    | DB decides (usually SEQUENCE)      | Medium     | Default fallback              |
| IDENTITY | @GeneratedValue(strategy=IDENTITY) | DB Auto-increment (MySQL IDENTITY) | Medium     | ■ Disables batch insert       |
| SEQUENCE | @GeneratedValue(strategy=SEQUENCE) | DB Sequence with allocationSize    | High       | ■ Used with allocationSize=50 |

| Strategy | Annotation                      | Mechanism                 | Throughput | ShopVerse             |
|----------|---------------------------------|---------------------------|------------|-----------------------|
| TABLE    | @GeneratedValue(strategy=TABLE) | Generator table with lock | Low        | ■ Avoid – contention  |
| UUID     | @UuidGenerator                  | Java UUID.randomUUID()    | High       | For external IDs only |

■ IDENTITY strategy disables Hibernate batch inserts because the DB generates IDs individually per row. Use SEQUENCE with allocationSize=50 for bulk insert performance.

## Bidirectional @OneToMany + @ManyToOne

```
@Entity @Table(name="orders")
public class Order extends BaseEntity {
    @ManyToOne(fetch=FetchType.LAZY)
    @JoinColumn(name="customer_id", nullable=false)
    private Customer customer;
    @OneToMany(mappedBy="order",
        cascade={PERSIST, MERGE},
        orphanRemoval=true,
        fetch=FetchType.LAZY)
    @OrderBy("sortOrder ASC")
    private List items = new ArrayList<>();
    // Bidirectional sync helpers (important!)
    public void addItem(OrderItem item) {
        items.add(item); item.setOrder(this); }
    public void removeItem(OrderItem item) {
        items.remove(item); item.setOrder(null); }
}
```

## Cascade Types Reference

| CascadeType | Effect on Children                   | ShopVerse Use                                |
|-------------|--------------------------------------|----------------------------------------------|
| PERSIST     | Save children when parent saved      | Order → OrderItems                           |
| MERGE       | Merge children when parent merged    | Order → OrderItems                           |
| REMOVE      | Delete children when parent deleted  | Handled by orphanRemoval=true (more precise) |
| REFRESH     | Refresh children from DB with parent | Rarely used                                  |
| DETACH      | Detach children when parent detached | Rarely needed                                |
| ALL         | All of the above                     | Avoid – REMOVE in ALL dangerous without care |

## Bidirectional vs Unidirectional Decision

| Scenario                                                               | Direction                 | Why                                                            |
|------------------------------------------------------------------------|---------------------------|----------------------------------------------------------------|
| Order has many OrderItems; always navigate from Order to OrderItems    | Unidirectional @OneToMany | Need traversal both ways; FK on OrderItem side                 |
| OrderItem references Product; never navigate from OrderItem to Product | Unidirectional @ManyToOne | Simple; avoid loading all items when loading product           |
| Customer has many Orders; navigate both ways                           | Bidirectional             | Customer dashboard lists orders; order detail shows customer   |
| Tag used on multiple Products                                          | Bidirectional @ManyToMany | Need to list all products for a tag and all tags for a product |

■ Always set mappedBy on the @OneToMany side. Without it, Hibernate creates an unneeded join table in addition to the FK column, wasting space and causing duplicate data.

## @ManyToMany with Join Table

```
@Entity public class Product extends BaseEntity {
    @ManyToMany(fetch=FetchType.LAZY)
    @JoinTable(
        name="product_tags",
        joinColumns=@JoinColumn(name="product_id"),
        inverseJoinColumns=@JoinColumn(name="tag_id"))
    private Set tags = new HashSet<>(); // Set prevents duplicate rows
}

@Entity public class Tag extends BaseEntity {
    @ManyToMany(mappedBy="tags")
    private Set products = new HashSet<>();
}
```

■ Use Set (not List) for @ManyToMany. With List, Hibernate DELETE ALL + INSERT ALL on any change to the collection, even adding a single tag. Set uses equals/hashCode to detect the specific change.

## @ManyToMany via Explicit Join Entity (recommended for extra columns)

```
// Instead of simple @ManyToMany join table, use explicit join entity when
// the join table has extra columns (e.g. priority, addedAt, addedBy)
@Entity @Table(name="product_categories")
public class ProductCategory extends BaseEntity {
    @ManyToOne @JoinColumn(name="product_id") private Product product;
    @ManyToOne @JoinColumn(name="category_id") private Category category;
    @Column(name="is_primary") private boolean primary; // extra column!
    @Column(name="sort_order") private int sortOrder; // extra column!
}
```

## @OneToOne Mapping Strategies

| Strategy                          | FK Location          | Lazy Possible?                             | Notes                                                      |
|-----------------------------------|----------------------|--------------------------------------------|------------------------------------------------------------|
| <b>@JoinColumn on owning side</b> | Owning entity table  | Requires bytecode instrumentation for lazy | Most common for straightforward                            |
| <b>mappedBy on inverse side</b>   | Inverse entity table | Yes (always lazy naturally)                | Useful when inverse is natural FK owner                    |
| <b>@PrimaryKeyJoinColumn</b>      | Shared PK            | Yes (always lazy)                          | Child PK = Parent PK; no separate FK column                |
| <b>@MapsId</b>                    | Shared PK from FK    | Yes                                        | Child shares PK with parent via @MapsId; cleanest approach |

```
@Entity public class Customer extends BaseEntity {
    @OneToOne(fetch=FetchType.LAZY, cascade=CascadeType.ALL, optional=false)
    @JoinColumn(name="wallet_id", unique=true)
    private CustomerWallet wallet;
}
```

## LAZY vs EAGER Comparison

| Strategy | When Loaded                 | Pros                         | Cons                                       | ShopVerse Default   |
|----------|-----------------------------|------------------------------|--------------------------------------------|---------------------|
| LAZY     | On first access of property | Avoids loading unused data   | LazyInitializationException out of the box | 1:1 relationships   |
| EAGER    | With parent entity          | Always-needed data available | Loads data even when not needed            | 1:1 & 1:n ShopVerse |

## N+1 Problem Illustration

```
// N+1 PROBLEM: 1 query for orders + N queries for items (one per order)
List orders = orderRepo.findAll(); // Query 1: SELECT * FROM orders
for (Order o : orders) {
    int count = o.getItems().size(); // Query 2..N+1: SELECT * FROM order_items WHERE order_id=?
}
// 100 orders → 101 queries total!
```

## N+1 Solutions Comparison

| Solution          | Syntax                                                   | SQL Generated          | Best For                                | Limitation                            |
|-------------------|----------------------------------------------------------|------------------------|-----------------------------------------|---------------------------------------|
| JOIN FETCH        | JOIN FETCH o.items in JPQL                               | 1 SQL JOIN             | Single relationship; control pagination | OOM with large results                |
| @EntityGraph      | @EntityGraph(attributePaths={"items", "items.customer"}) | 1 SQL JOIN             | Multiple relationships                  | Cartesian product if many collections |
| @BatchSize        | @BatchSize(size=50) on collection                        | 1 + ceil(N/50) queries | Many collections; large data            | Extra queries but no cartesian        |
| @Fetch(SUBSELECT) | @Fetch(FetchMode.SUBSELECT)                              | 2 queries              | Alternative to batch                    | Full result set in subselect          |

```
// Solution 1: JOIN FETCH (specify DISTINCT to avoid duplicates)
@Query("SELECT DISTINCT o FROM Order o JOIN FETCH o.items WHERE o.customerId=:id")
List findWithItemsByCustomer(@Param("id") long customerId);

// Solution 2: @EntityGraph (declarative; on repository method)
@EntityGraph(attributePaths={"items", "items.product"})
List findByCustomerId(long customerId);

// Solution 3: @BatchSize on collection (prevents OOM with pagination)
@OneToMany(mappedBy="order", fetch=LAZY)
@BatchSize(size=50)
private List items = new ArrayList<>();

■ JOIN FETCH with Pageable triggers HHH90003004: Hibernate loads ALL results in memory then paginates. Fix: Use @BatchSize, or two-query approach (query IDs paginated first, then fetch entities by IDs).
```

## JPQL vs SQL vs Criteria API Comparison

| Feature                | JPQL                  | Native SQL                 | Criteria API              |
|------------------------|-----------------------|----------------------------|---------------------------|
| <b>Works on</b>        | Entity names & fields | Table & column names       | Entity names (type-safe)  |
| <b>Type-safe</b>       | No (String)           | No (String)                | Yes (compile-time)        |
| <b>DB portable</b>     | Yes                   | No (dialect)               | Yes                       |
| <b>Complex queries</b> | Moderate              | Full SQL power             | Moderate; verbose         |
| <b>Dynamic queries</b> | Limited               | Via StringBuilder (unsafe) | Yes – composed predicates |
| <b>ShopVerse usage</b> | Most @Query methods   | Reporting, window funcs    | ProductSpecification      |

```
// JPQL – most readable for straightforward queries
@Query("SELECT p FROM Product p WHERE p.categoryId=:catId AND p.active=true ORDER BY p.name")
List findActive(@Param("catId") long catId);

// JPQL with JOIN – explicit join on non-FK field
@Query("SELECT o FROM Order o JOIN o.items i WHERE i.productId=:pid AND o.status=:st")
List findOrdersContaining(@Param("pid") long pid, @Param("st") OrderStatus st);

// Native SQL – window function for ranking (not possible in JPQL)
@Query(value="
SELECT p.*, RANK() OVER (PARTITION BY p.category_id ORDER BY p.sales_count DESC) AS rank
FROM products p WHERE p.is_active = true", nativeQuery=true)
List findRankedProducts();
```

## Interface Projections

```
// Interface projection – DB sends only needed columns
public interface OrderSummary {
    Long getId();
    BigDecimal getTotal();
    OrderStatus getStatus();
    @Value("#{target.firstName + ' ' + target.lastName}") // SpEL
    String getCustomerName();
}

// Repository returns projection
List findByCustomerId(long customerId);

// Class (DTO) projection – constructor expression
@Query("SELECT new com.shopverse.dto.OrderSummaryDto(o.id, o.total, o.status) FROM Order o
WHERE o.customerId=:id")
List findSummariesByCustomer(@Param("id") long id);
```

## Optimistic vs Pessimistic Locking

| Aspect                    | Optimistic (@Version)                 | Pessimistic (SELECT FOR UPDATE)           |
|---------------------------|---------------------------------------|-------------------------------------------|
| <b>Mechanism</b>          | Version column checked on UPDATE      | Row-level DB lock held during transaction |
| <b>Conflict detection</b> | At commit time                        | At read time (may block)                  |
| <b>Throughput</b>         | High (no locks held)                  | Lower (writers blocked)                   |
| <b>Deadlock risk</b>      | None                                  | Yes – order locks carefully               |
| <b>Best for</b>           | Read-heavy, rare conflicts            | Write-heavy, conflict expected            |
| <b>ShopVerse</b>          | @Version on BaseEntity (all entities) | FlashSaleService: PESSIMISTIC_WRITE       |
| <b>Exception</b>          | OptimisticLockException → 409         | PessimisticLockException → 503/retry      |

```
// Optimistic locking with retry
@Retryable(value=OptimisticLockException.class, maxAttempts=3,
backoff=@Backoff(delay=100, multiplier=2))
@Transactional public void updateStock(long productId, int delta) {
    InventoryItem item = repo.findById(productId).orElseThrow();
    item.setStock(item.getStock() + delta);
    repo.save(item); // Hibernate: UPDATE ... WHERE version=? → throws if concurrent mod
}

// Pessimistic locking for flash sale
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT i FROM InventoryItem i WHERE i.productId=:id")
Optional findByIdForUpdate(@Param("id") long id);
```

## Isolation Levels Reference

| Level                           | Dirty Read | Non-Repeatable | Phantom Read | ShopVerse Use                         |
|---------------------------------|------------|----------------|--------------|---------------------------------------|
| <b>READ UNCOMMITTED</b>         | Possible   | Possible       | Possible     | Never – too risky                     |
| <b>READ COMMITTED (default)</b> | No         | Possible       | Possible     | Default for most service methods      |
| <b>REPEATABLE READ</b>          | No         | No             | Possible     | Flash sale inventory check            |
| <b>SERIALIZABLE</b>             | No         | No             | No           | Critical financial aggregation (rare) |



## Cache Layers Overview

### L1 Cache (Session / Persistence Context) – always enabled

Scope: single EntityManager / transaction

em.find(Order.class, id) returns same object instance if called twice in sa

Cleared on em.clear() or end of transaction

### L2 Cache (Second-Level Cache) – optional; cross-session

Providers: Ehcache, Caffeine, Redis (ShopVerse uses Redis)

@Cache(usage=READ\_WRITE) on @Entity or @OneToMany collection

Stores entity data (not instances) keyed by PK

### Query Cache – caches result sets (list of entity IDs)

hibernate.cache.use\_query\_cache=true

@QueryHint(name=HINT\_CACHEABLE, value="true") pairs with L2 for actual data

## L2 Cache Concurrency Strategies

| Strategy             | Read   | Write                                                                              | Suitable For                                      |
|----------------------|--------|------------------------------------------------------------------------------------|---------------------------------------------------|
| READ_ONLY            | Cached | Throws on update                                                                   | Immutable data: currencies, countries, categories |
| READ_WRITE           | Cached | Soft-lock + update after tx                                                        | Frequently read, occasionally updated: products   |
| NONSTRICT_READ_WRITE | Cached | Update without lock (brief stale <del>possible</del> ), low-write, stale tolerable |                                                   |
| TRANSACTIONAL        | Cached | JTA-aware update                                                                   | Full consistency; JTA env required (rare)         |

```
# application.yml - L2 cache with Redis
spring.jpa.properties:
hibernate.cache.use_second_level_cache: true
hibernate.cache.region.factory_class: org.hibernate.cache.redis.hibernate.RedisRegionFactory
hibernate.cache.use_query_cache: true
hibernate.cache.default_cache_concurrency_strategy: READ_WRITE
@Entity @Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Product extends BaseEntity { ... }
@OneToMany(mappedBy="product") @Cache(usage = READ_WRITE)
private List reviews;
```

## Migration File Naming

| File Pattern                       | Type       | When Runs                                          |
|------------------------------------|------------|----------------------------------------------------|
| <b>V1__create_core_tables.sql</b>  | Versioned  | Once; checksummed; never modified after apply      |
| <b>V2__add_product_indexes.sql</b> | Versioned  | Once; in strict version order                      |
| <b>R__refresh_search_view.sql</b>  | Repeatable | Every time checksum changes                        |
| <b>B1__baseline.sql</b>            | Baseline   | Only for pre-existing DBs (snapshot current state) |

```
# application.yml
spring.flyway:
  enabled: true
  locations: classpath:db/migration
  baseline-on-migrate: true
  validate-on-migrate: true
  out-of-order: false
-- V1__create_core_tables.sql
CREATE TABLE orders (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  customer_id BIGINT NOT NULL REFERENCES customers(id),
  status VARCHAR(30) NOT NULL DEFAULT 'PENDING',
  total_amount NUMERIC(19,4) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  version INTEGER NOT NULL DEFAULT 0,
  CONSTRAINT chk_total CHECK (total_amount >= 0)
);
CREATE INDEX idx_orders_customer_status ON orders(customer_id, status, created_at DESC);
```

## Flyway Rules

| Rule                                          | Rationale                                                                |
|-----------------------------------------------|--------------------------------------------------------------------------|
| <b>Never modify a versioned migration</b>     | Flyway validates checksum on startup; modification → ValidationException |
| <b>Use IF NOT EXISTS / IF EXISTS</b>          | Idempotent scripts; safer on re-run                                      |
| <b>Add indexes in separate migrations</b>     | Isolated; clear which migration added each index                         |
| <b>Keep migrations small</b>                  | Easier rollback analysis; faster apply on restart                        |
| <b>Test migrations against prod-like data</b> | Column rename can be slow on 100M+ rows – use shadow column + backfill   |

## Batch Insert Configuration

```
# application.yml
spring.jpa.properties:
hibernate.jdbc.batch_size: 50
hibernate.order_inserts: true
hibernate.order_updates: true
hibernate.batch_versioned_data: true
hibernate.generate_statistics: true # dev/staging only
// Batch insert pattern
@Transactional public void importProducts(List raw) {
for (int i = 0; i < raw.size(); i++) {
em.persist(new Product(raw.get(i)));
if (i % 50 == 0) { em.flush(); em.clear(); } // prevent heap growth
}
}
```

## Index Design Guidelines

| Table           | Index                                                   | Type      | Reason                                   |
|-----------------|---------------------------------------------------------|-----------|------------------------------------------|
| orders          | (customer_id, status, created_at)                       | Composite | Filter customer+status; sort by date     |
| orders          | (status) WHERE status NOT IN ('DELIVERED', 'CANCELLED') | Partial   | Filter active orders only; smaller index |
| order_items     | (order_id)                                              | FK        | Join Order → Items                       |
| products        | (category_id, is_active, price)                         | Composite | Catalog browse with price filter         |
| products        | GIN on to_tsvector(name  description)                   | Full-text | Pre-ES full-text search                  |
| customers       | UNIQUE (email)                                          | Unique    | Login lookup; uniqueness enforcement     |
| inventory_items | (product_id)                                            | FK        | Join Product → Inventory                 |

## Hibernate Statistics Thresholds

| Metric                         | Warning Threshold      | Fix                                              |
|--------------------------------|------------------------|--------------------------------------------------|
| getQueryExecutionCount()       | > 10 per request       | N+1 – add JOIN FETCH or @BatchSize               |
| getEntityInsertCount()         | > 50 per request       | Use saveAll() with batch_size=50                 |
| getSecondLevelCacheMissCount() | > 80% miss ratio       | L2 cache not warming; check @Cache config        |
| getCollectionLoadCount()       | > getEntityLoadCount() | Collection-heavy access pattern; check FetchType |

Hibernate Envers automatically tracks changes (INSERT/UPDATE/DELETE) to @Audited entities, storing each revision in separate \_AUD tables. ShopVerse uses this for the Order and Product audit trail.

```
// Enable Envers
@Entity @Audited // tracks all changes to this entity
public class Order extends BaseEntity { ... }
@Entity @Audited(withModifiedFlag=true) // also records which columns changed
public class Product extends BaseEntity { ... }

// Envers creates:
// REVINFO table: revtstp (timestamp), id (revision number)
// orders_AUD: all Order columns + revtype (0=ADD, 1=MOD, 2=DEL), rev (FK→REVINFO)

// Query audit history
@Autowired AuditReader auditReader;

// Get all revisions of an Order
List revisions = auditReader.getRevisions(Order.class, orderId);

// Get state at specific revision
Order atRevision = auditReader.find(Order.class, orderId, revisions.get(0));

// Get all changes between two timestamps
AuditQuery q = auditReader.createQuery()
    .forRevisionsOfEntity(Order.class, false, true)
    .add(AuditEntity.id().eq(orderId))
    .add(AuditEntity.revisionProperty("timestamp").between(from, to));
```

## ProductSpecification – Dynamic Search

```
public class ProductSpecification {
    public static Specification withCategoryId(Long catId) {
        return (root, q, cb) -> {
            catId == null ? cb.conjunction() : cb.equal(root.get("categoryId"), catId);
        }
    }
    public static Specification withPriceBetween(BigDecimal min, BigDecimal max) {
        return (root, q, cb) -> {
            if (min == null && max == null) return cb.conjunction();
            if (max == null) return cb.ge(root.get("price"), min);
            if (min == null) return cb.le(root.get("price"), max);
            return cb.between(root.get("price"), min, max);
        };
    }
    public static Specification withKeyword(String kw) {
        return (root, q, cb) -> kw == null || kw.isBlank() ? cb.conjunction() :
        cb.like(cb.lower(root.get("name")), "%" + kw.toLowerCase() + "%");
    }
    public static Specification isActive() {
        return (root, q, cb) -> cb.isTrue(root.get("active"));
    }
}

// Usage in service
Specification spec = Specification
    .where(ProductSpecification.isActive())
    .and(ProductSpecification.withCategoryId(criteria.getCategoryId()))
    .and(ProductSpecification.withPriceBetween(criteria.getMinPrice(), criteria.getMaxPrice()))
    .and(ProductSpecification.withKeyword(criteria.getKeyword()));
Page page = productRepo.findAll(spec,
    PageRequest.of(criteria.getPage(), criteria.getSize(), Sort.by("name")));
```

## JPA Inheritance Strategies Comparison

| Strategy               | Annotation                             | Tables                   | NULL Columns                  | Polymorphic Q                 |                       |
|------------------------|----------------------------------------|--------------------------|-------------------------------|-------------------------------|-----------------------|
| <b>SINGLE_TABLE</b>    | @Inheritance(strategy=SINGLE_TABLE)    | 1 table for all types    | Many NULLs (non-type columns) | 1 query (fastest)             | PaymentMethod         |
| <b>JOINED</b>          | @Inheritance(strategy=JOINED)          | 1 parent + 1 per subtype | None (columns in right table) | 1 query per subtype (slowest) | Not used (pre-Java 5) |
| <b>TABLE_PER_CLASS</b> | @Inheritance(strategy=TABLE_PER_CLASS) | 1 table per class        | None                          | UNION ALL (slowest)           | Never in Shop         |

```

@Entity @Inheritance(strategy=InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name="payment_type", discriminatorType=DiscriminatorType.STRING)
public abstract class PaymentMethod extends BaseEntity {
    @Column(name="customer_id") private long customerId;
}

@Entity @DiscriminatorValue("CARD")
public class CardPaymentMethod extends PaymentMethod {
    private String last4; private String brand; private int expiryYear;
}

@Entity @DiscriminatorValue("PAYPAL")
public class PayPalPaymentMethod extends PaymentMethod {
    private String paypalEmail;
}

```

■ **SINGLE\_TABLE** is the fastest for polymorphic queries (no JOINS), but tolerates NULL columns for type-specific fields. Use @Column(nullable=true) and NOT NULL constraints at the Java level via @NotNull in the concrete class.

## ■ Do

- ✓ Default all relationships to LAZY; JOIN FETCH only when you know you need the data.
- ✓ Use @Version (optimistic locking) on all entities via BaseEntity.
- ✓ Use SEQUENCE generator with allocationSize=50 for high-throughput inserts.
- ✓ Use Set (not List) for @ManyToMany to prevent DELETE ALL + INSERT ALL.
- ✓ Add @BatchSize(size=50) on lazy collections to handle unavoidable N+1.
- ✓ Never modify a versioned Flyway migration – create a new version for changes.
- ✓ Enable hibernate.jdbc.batch\_size=50 for bulk insert operations.
- ✓ Use interface projections for read-only APIs – avoids hydrating full entities.
- ✓ Use Specification API for complex dynamic queries instead of many findByX methods.
- ✓ Add partial indexes (WHERE clause) for queries on a subset of rows (e.g., active orders only).
- ✓ Use @Audited for entities where you need full change history (orders, products).

## ■ Anti-Patterns

- FetchType.EAGER – loads data even when not needed; causes cartesian products with multiple collections.
- CascadeType.ALL without orphanRemoval – leaves orphan rows in DB on collection modification.
- JPQL with Pageable + JOIN FETCH – loads all results in memory first (HHH90003004).
- Bidirectional @OneToMany without mappedBy – creates unintended join table.
- Calling em.flush() inside a loop without em.clear() – heap pressure from large persistence context.
- FetchType.LAZY on @OneToOne without bytecode enhancement – Hibernate loads eagerly anyway.
- IDENTITY generation strategy – disables Hibernate batch inserts.
- Long-running transactions holding pessimistic locks – causes contention and timeouts under load.
- Ignoring Hibernate statistics – missed N+1 issues and cache misses go undetected.

| Sub-Chapter       | Key Rule                       | One-Liner                                                          |
|-------------------|--------------------------------|--------------------------------------------------------------------|
| 04-01 Mapping     | SEQUENCE allocationSize=50     | Reduces DB round-trips for ID generation; enables batch            |
| 04-02 @OneToMany  | mappedBy + orphanRemoval       | mappedBy avoids join table; orphanRemoval cleans child rows        |
| 04-03 @ManyToMany | Use Set                        | List causes DELETE ALL + INSERT ALL on any collection change       |
| 04-04 N+1         | @BatchSize(50) on collection   | Prevents N+1 without cartesian product on large datasets           |
| 04-05 JPQL        | Interface projection           | SELECT only needed columns; Hibernate does not hydrate full entity |
| 04-06 Locking     | @Version = optimistic          | Compare version on UPDATE; PESSIMISTIC_WRITE for flash sale        |
| 04-07 Cache       | READ_WRITE on Product          | L2 cache for frequently read; invalidate on update                 |
| 04-08 Flyway      | Never modify applied migration | Create V{n+1} for any change; checksum validated on startup        |
| 04-09 Perf        | batch_size=50 + saveAll()      | Groups INSERTs; avoids individual round-trips per row              |
| 04-10 Envers      | @Audited on Order/Product      | Full audit trail; query with AuditReader                           |
| 04-11 Spec        | Specification.where().and()    | Null-safe predicate composition; reusable across queries           |
| 04-12 Inheritance | SINGLE_TABLE default           | Fastest polymorphic query; no JOINS needed                         |

## Interview Quick-Check

| Question                         | Concise Answer                                                                                     |
|----------------------------------|----------------------------------------------------------------------------------------------------|
| @OneToMany without mappedBy?     | Hibernate creates an extra join table even though there's a FK on the child. Always add mappedBy   |
| How does @Version work?          | Hibernate adds AND version=? to UPDATE. If 0 rows updated, version was changed by another tx       |
| N+1 problem and best fix?        | 1 query for N parents + N child queries. For lists: @BatchSize. For single entity: JOIN FETCH or @ |
| IDENTITY vs SEQUENCE generation? | IDENTITY: DB assigns ID per row – Hibernate can't batch. SEQUENCE with allocationSize=50: pr       |
| Flyway validate failure?         | Checksum of already-applied migration changed. Fix: create a new migration; NEVER edit applied     |
| L1 vs L2 cache?                  | L1: per-transaction/EntityManager – always on. L2: cross-transaction, optional, needs provider (Re |